National University of Sciences & Technology
School of Electrical Engineering and Computer Science
Department of Computing

CS 417: Large Language Model (2+1) **Spring 2026**

Assignment 1: Build an Urdu BPE Tokenizer from Scratch

CLO 3: *Develop and apply LLM-based solutions for practical NLP problems using suitable interaction and adaptation strategies.*

Overview

You will build a byte-pair encoding (BPE) tokenizer for Urdu from an empty folder. You collect the text, write the algorithm yourself, and train a vocabulary of 50,000 tokens.

The corpus and tokenizer you produce are the inputs to the next assignment, where you train a GPT-style model on them. Every weakness in the data or the tokenizer carries forward, so build both with care.

GPT-2's tokenizer was trained mostly on English text, so it breaks Urdu words into small byte fragments. You will try to train a better version for Urdu in next assignment.

This assignment has three tasks:

1. **Part A - Corpus.** Collect and clean at least 500 MB of Urdu text.
2. **Part B - Tokenizer.** Implement byte-level BPE without any tokenizer library and train it to 50,000 tokens.
3. **Part C - Analysis.** Study what your tokenizer learned and compare it with existing tokenizers.

Learning outcomes

After this assignment you can:

- Explain how BPE builds a vocabulary, and why a byte-level tokenizer never meets an unknown token.
- Build and document a text corpus: sourcing, cleaning, deduplication and splits.
- Handle Unicode correctly, including UTF-8 bytes and the code point problems specific to Urdu.
- Judge a tokenizer with numbers instead of impressions.

Part A: Build the corpus

Deliver at least 500 MB of cleaned Urdu text in UTF-8, split into train, validation and test files. At about two bytes per Urdu letter, that is roughly 50 to 60 million words.

The next assignment needs enough text to train a language model so you can aim for 1 GB if you can.

Sources

- Combine at least three sources from at least two genres, such as news, encyclopedia, literature and blogs.
- Collect at least 50 MB yourself by scraping or through an API and give evidence. The rest may come from published corpora.

- Record the licence or terms of use of every source.

Starting points:

Source	What it offers	Watch out for
Urdu Wikipedia dumps	Encyclopedia articles	Wiki markup must be stripped
UFAL Urdu Monolingual Corpus	5.46 million sentences of plain text, 213 MB compressed	Released in 2014; read its licence
CC-100 and OSCAR	Urdu text from Common Crawl web crawls	Noisy; needs strong cleaning and deduplication
Urdu News Dataset	About 1 million news stories	One genre only
Makẖzan	6.26 million words of edited Urdu	Small; its text may not be republished
News sites, blogs and public-domain books	Fresh text for your own collection	Follow the collection rules below

A longer list of Urdu datasets is in awesome-urdu.

Collection rules

- Obey each site's `robots.txt` and terms of use. Skip anything behind a login or paywall.
- Collect no personal data: no private messages and no social media profiles.
- Keep the corpus inside the course. Do not publish text you have no right to redistribute.

Cleaning pipeline

1. **Extract** plain text. Strip HTML, wiki markup, menus, adverts and share buttons.
2. **Normalise** Unicode with your own `normalize(text)` function, described below.
3. **Filter** out lines that are mostly not Urdu. Keep natural code-mixing, such as English words inside Urdu sentences.
4. **Deduplicate.** Remove exact duplicate documents and boilerplate lines that repeat across pages.
5. **Split** by document, not by line: 98% train, 1% validation, 1% test, with a fixed random seed.

Save the result as `train.txt`, `val.txt` and `test.txt`, with `<|endoftext|>` after each document.

Urdu normalization

The same Urdu word is often typed with different code points that look identical on screen. Without normalization, your tokenizer learns every variant as a separate token. Few examples of normalization are mentioned below. You can explore more and include.

Problem	Code points	What to do
Arabic yeh typed for Urdu yeh	Arabic yeh ي (U+064A) and alef maksura ى (U+0649), in place of Urdu yeh ی (U+06CC)	Map to U+06CC
Arabic kaf typed for Urdu kaf	Arabic kaf ك (U+0643), in place of Urdu kaf ک (U+06A9)	Map to U+06A9

Problem	Code points	What to do
Heh variants	Arabic heh ھ (U+0647), gol heh ہ (U+06C1), do-chashmi heh ھ (U+06BE)	Map U+0647 to U+06C1; keep U+06C1 and U+06BE apart, they are different letters
Tatweel	U+0640	Remove
Invisible characters	Zero-width joiner, direction marks, byte-order mark, ZWNJ (U+200C)	Remove; for ZWNJ, which Urdu uses on purpose, decide and justify
Digits	U+0660 to U+0669, U+06F0 to U+06F9, and ASCII 0 to 9	Pick one convention
Whitespace	Non-breaking spaces, runs of spaces, stray line breaks	Collapse to single spaces and newlines

Data card

Submit a one-page `DATA_CARD.md` with:

- A table of sources: URL, licence, collection date, raw size and cleaned size.
- Corpus totals: MB, documents, words and unique words.
- Each cleaning step and how much text it removed.
- Known gaps and biases, such as a genre that dominates.

Part B: Implement BPE from scratch

Write a byte-level BPE tokenizer in plain Python and train it on your `train.txt` to exactly 50,000 tokens.

Specification

- **Base vocabulary.** The 256 byte values, with IDs 0 to 255. Urdu letters take two bytes each in UTF-8, so the four-letter word کتاب starts as eight tokens.
- **Special token.** `<|endoftext|>` has ID 256. It is never split and never takes part in a merge.
- **Merges.** IDs 257 to 49,999, in the order learned. That is 49,743 merges.
- **Pre-tokenisation.** Split the text into words with a regular expression before counting pairs, so that merges never cross a word boundary. A leading space belongs to the word that follows it, as in GPT-2.
- **Tie-breaking.** When two pairs have the same count, merge the greater pair, comparing the pairs as tuples of `bytes`. This makes training deterministic.
- **Lossless.** `decode(encode(text)) == text` for every Unicode string, including emoji, English, newlines and the empty string. Normalisation is a separate step and stays outside the tokenizer.

Pre-tokenisation for Urdu

GPT-2's pattern, shown below, does not work for Urdu. It treats combining marks as punctuation, so it cuts the word اُردُو into five pieces at its diacritics. Zero-Width Non-Joiner (ZWNJ) splits a word in the same way.

```
's|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+
```

Design your own pattern that keeps Urdu words whole. Use the `regex` module, which supports `\p{L}`, `\p{M}` and `\p{N}`, and explain your pattern in the report.

Build it in two stages

1. **Reference version.** Recount every pair after every merge. Test it on the sanity check below and on a 1 MB sample. It is too slow for the full corpus.
2. **Fast version.** Add the three optimisations below. Confirm that it learns the same merges as the reference version on the 1 MB sample.

The three optimisations:

- Count each distinct word once, and weight its pairs by the word's frequency.
- After a merge, update counts only for the words that contained the merged pair. Keep an index from each pair to the words that contain it.
- When encoding the corpus, cache the token IDs of every word already encoded.

The fast version must finish training on your full `train.txt`. A good implementation generally takes hours, not days. Report the wall-clock time and the machine you used.

Encode the corpus

Encode `train.txt` and `val.txt` with the final tokenizer. Save the IDs as `train.bin` and `val.bin`, flat NumPy arrays of type `uint16`, which is wide enough for 50,000 IDs. The next assignment reads these files directly.

Part C: Analyse your tokenizer

Answer five questions in a report of at most four pages. Give each answer a table or plot and two or three sentences of interpretation.

1. **What did the merges learn?** List merges 1 to 50, and every 5,000th merge after that, as decoded text. Which early merges rebuild a two-byte letter, and which produce a token that is not valid UTF-8 on its own? At which merge does the first whole word appear? Show your 20 longest tokens.
2. **How does it compare with existing tokenizers?** Measure tokens per word for your tokenizer, GPT-2's tokenizer and one modern multilingual tokenizer. Use `test.txt` and seven inputs you pick by hand: news, poetry, Urdu with English words, Roman Urdu, numbers and dates, text with diacritics, and personal names. The `tiktoken` library provides GPT-2's encoding as `gpt2` and a newer one as `o200k_base`.
3. Report three inputs that your tokenizer handles badly, and explain each.

Deliverables

Submit one archive, plus a shared-drive link for the files that are too large to upload.

In the archive:

- **Code:** `normalize.py`, `bpe.py` containing `BPETokenizer`, `train_tokenizer.py`, `encode_corpus.py`, and your collection and cleaning scripts.
- **Tokenizer:** the saved tokenizer file with all 49,743 merges in order.
- **Data card:** `DATA_CARD.md`.
- **Report:** `report.pdf`, at most four pages.

- **README:** the exact commands that rebuild the tokenizer from `train.txt`.

By link:

- `train.txt`, `val.txt` and `test.txt`.
- `train.bin` and `val.bin`.

Rules

Every line of the tokenizer and the normaliser must be your own work.

- **Allowed in your implementation:** the Python standard library, `regex`, `numpy`, `tqdm`, and collection tools such as `requests` and `beautifulsoup4`.
- **Not allowed in your implementation:** any tokenizer library or published BPE code, including `tokenizers`, `sentencepiece`, `tiktoken` and `minbpe`. ***Urdu NLP libraries may not do your normalisation for you.***
- **Allowed for comparison in Part C only:** `tiktoken` and `tokenizers`.
- You may be asked to explain any part of your code in a short viva.